Hand-Sketch Project Report

1. Introduction

The **Hand-Sketch** project is a virtual drawing tool that uses hand gestures to control a brush on a digital canvas. It leverages hand detection and tracking using a camera to recognize finger movements and gestures. This allows users to draw, erase, and clear a virtual whiteboard using natural hand gestures, enhancing user interaction with simple, intuitive controls.

Key Features:

- Hand gesture recognition for drawing.
- Adjustable brush sizes and color selection.
- Eraser tool and clear canvas button.
- Color palette with dynamic color preview.
- User-friendly interface (GUI).

Tech Stack:

- **Python**: The programming language used for development.
- **OpenCV**: Handles video capture and image processing.
- **Mediapipe**: Facilitates hand detection and gesture recognition.
- **NumPy**: Assists in creating and manipulating the drawing canvas.

---

2. Hand Detection and Gesture Recognition

The core functionality of the project revolves around recognizing hand gestures in real-time using a camera. The `HandTracker` class in **`handtracker.py`** performs the heavy lifting by using **Mediapipe** for hand landmark detection.

Code Explanation:

`HandTracker` Class

This class tracks the hand's position, detects gestures, and extracts finger states. Here's the breakdown of the class's main methods:

```
class HandTracker():
    def __init__(self, mode=False, maxHands=2, detectionCon=0.5, trackCon=0.5):
        self.mode = mode
        self.maxHands = maxHands
```

```
        self.detectionCon = detectionCon
        self.trackCon = trackCon

        self.mpHands = mp.solutions.hands
        self.hands = self.mpHands.Hands(self.mode, self.maxHands, self.detectionCon,
self.trackCon)
        self.mpDraw = mp.solutions.drawing_utils
```

- **`__init__()`**: Initializes the hand detection model. You can control detection mode, the number of hands, and detection confidence.
- **`findHands()`**: Converts the frame to RGB, detects hand landmarks, and optionally draws them on the frame.

```
    def findHands(self, img, draw=True):
        imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        self.results = self.hands.process(imgRGB)

        if self.results.multi_hand_landmarks:
            for handLm in self.results.multi_hand_landmarks:
                if draw:
                    self.mpDraw.draw_landmarks(img, handLm, self.mpHands.HAND_CONNECTIONS)
        return img
```

- **`getPostion()`**: Extracts the coordinates of each finger landmark from the detected hand(s).

```
    def getPostion(self, img, handNo = 0, draw=True):
        lmList =[]
        if self.results.multi_hand_landmarks:
            myHand = self.results.multi_hand_landmarks[handNo]
            for lm in myHand.landmark:
                h, w, c = img.shape
                cx, cy = int(lm.x*w), int(lm.y*h)
                lmList.append((cx, cy))

                if draw:
                    cv2.circle(img, (cx, cy), 5, (255,0,255), cv2.FILLED)
        return lmList
```

- **`getUpFingers()`**: Detects which fingers are raised, facilitating gesture control (e.g., drawing with the index finger).

```
    def getUpFingers(self, img):
        pos = self.getPostion(img, draw=False)
        self.upfingers = []
        if pos:
            #thumb
            self.upfingers.append((pos[4][1] < pos[3][1] and (pos[5][0]-pos[4][0]> 10)))
            #index
```

```
            self.upfingers.append((pos[8][1] < pos[7][1] and pos[7][1] < pos[6][1]))
            #middle
            self.upfingers.append((pos[12][1] < pos[11][1] and pos[11][1] < pos[10][1]))
            #ring
            self.upfingers.append((pos[16][1] < pos[15][1] and pos[15][1] < pos[14][1]))
            #pinky
            self.upfingers.append((pos[20][1] < pos[19][1] and pos[19][1] < pos[18][1]))
        return self.upfingers
```

This method evaluates whether each finger is raised based on the hand landmarks' y-coordinates.

---

3. Main Application

The main application **`main.py`** ties everything together. It continuously captures video from the camera, uses the hand detection model to track gestures, and provides the GUI where users can draw on the canvas.

How it Works:

- The application captures video frames using **OpenCV** and processes them with **Mediapipe** to detect hand gestures.
- Depending on the finger positions, the brush draws or erases lines on a virtual canvas created using **NumPy** arrays.
- The brush color, size, and mode (draw/erase) can be dynamically changed using gestures.

Code Overview:

The `main.py` file handles:

- Capturing the video feed.
- Detecting hand gestures and updating the virtual canvas.
- Adjusting the brush's size and color based on user input.

```
from handTracker import *
import cv2
import mediapipe as mp
import numpy as np
import random

class ColorRect():
    def __init__(self, x, y, w, h, color, text='', alpha = 0.5):
        self.x = x
        self.y = y
        self.w = w
```

```
        self.h = h
        self.color = color
        self.text=text
        self.alpha = alpha


    def drawRect(self, img, text_color=(255,255,255), fontFace=cv2.FONT_HERSHEY_SIMPLEX,
fontScale=0.8, thickness=2):
        #draw the box
        alpha = self.alpha
        bg_rec = img[self.y : self.y + self.h, self.x : self.x + self.w]
        white_rect = np.ones(bg_rec.shape, dtype=np.uint8)
        white_rect[:] = self.color
        res = cv2.addWeighted(bg_rec, alpha, white_rect, 1-alpha, 1.0)

        # Putting the image back to its position
        img[self.y : self.y + self.h, self.x : self.x + self.w] = res

        #put the letter
        tetx_size = cv2.getTextSize(self.text, fontFace, fontScale, thickness)
        text_pos = (int(self.x + self.w/2 - tetx_size[0][0]/2), int(self.y + self.h/2 +
tetx_size[0][1]/2))
        cv2.putText(img, self.text,text_pos , fontFace, fontScale,text_color, thickness)


    def isOver(self,x,y):
        if (self.x + self.w > x > self.x) and (self.y + self.h> y >self.y):
            return True
        return False


#initilize the habe detector
detector = HandTracker(detectionCon=1)

#initilize the camera
cap = cv2.VideoCapture(0)
cap.set(3, 1280)
cap.set(4, 720)

# creating canvas to draw on it
canvas = np.zeros((720,1280,3), np.uint8)

# define a previous point to be used with drawing a line
px,py = 0,0
#initial brush color
color = (255,0,0)
#####
brushSize = 5
eraserSize = 20
```

```
####

############ creating colors ########
# Colors button
colorsBtn = ColorRect(200, 0, 100, 100, (120,255,0), 'Colors')

colors = []
#random color
b = int(random.random()*255)-1
g = int(random.random()*255)
r = int(random.random()*255)
print(b,g,r)
colors.append(ColorRect(300,0,100,100, (b,g,r)))
#red
colors.append(ColorRect(400,0,100,100, (0,0,255)))
#blue
colors.append(ColorRect(500,0,100,100, (255,0,0)))
#green
colors.append(ColorRect(600,0,100,100, (0,255,0)))
#yellow
colors.append(ColorRect(700,0,100,100, (0,255,255)))
#erase (black)
colors.append(ColorRect(800,0,100,100, (0,0,0), "Eraser"))

#clear
clear = ColorRect(900,0,100,100, (100,100,100), "Clear")

########## pen sizes #######
pens = []
for i, penSize in enumerate(range(5,25,5)):
    pens.append(ColorRect(1100,50+100*i,100,100, (50,50,50), str(penSize)))

penBtn = ColorRect(1100, 0, 100, 50, color, 'Pen')

# white board button
boardBtn = ColorRect(50, 0, 100, 100, (255,255,0), 'Board')

#define a white board to draw on
whiteBoard = ColorRect(50, 120, 1020, 580, (255,255,255),alpha = 0.6)

coolingCounter = 20
hideBoard = True
hideColors = True
hidePenSizes = True

while True:

    if coolingCounter:
```

```
        coolingCounter -=1
        #print(coolingCounter)

    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.resize(frame, (1280, 720))
    frame = cv2.flip(frame, 1)

    detector.findHands(frame)
    positions = detector.getPostion(frame, draw=False)
    upFingers = detector.getUpFingers(frame)

    if upFingers:
        x, y = positions[8][0], positions[8][1]
        if upFingers[1] and not whiteBoard.isOver(x, y):
            px, py = 0, 0

            ##### pen sizes ######
            if not hidePenSizes:
                for pen in pens:
                    if pen.isOver(x, y):
                        brushSize = int(pen.text)
                        pen.alpha = 0
                    else:
                        pen.alpha = 0.5

            ####### chose a color for drawing #######
            if not hideColors:
                for cb in colors:
                    if cb.isOver(x, y):
                        color = cb.color
                        cb.alpha = 0
                    else:
                        cb.alpha = 0.5

                #Clear
                if clear.isOver(x, y):
                    clear.alpha = 0
                    canvas = np.zeros((720,1280,3), np.uint8)
                else:
                    clear.alpha = 0.5

            # color button
            if colorsBtn.isOver(x, y) and not coolingCounter:
                coolingCounter = 10
                colorsBtn.alpha = 0
                hideColors = False if hideColors else True
```

```
                colorsBtn.text = 'Colors' if hideColors else 'Hide'
            else:
                colorsBtn.alpha = 0.5

            # Pen size button
            if penBtn.isOver(x, y) and not coolingCounter:
                coolingCounter = 10
                penBtn.alpha = 0
                hidePenSizes = False if hidePenSizes else True
                penBtn.text = 'Pen' if hidePenSizes else 'Hide'
            else:
                penBtn.alpha = 0.5


            #white board button
            if boardBtn.isOver(x, y) and not coolingCounter:
                coolingCounter = 10
                boardBtn.alpha = 0
                hideBoard = False if hideBoard else True
                boardBtn.text = 'Board' if hideBoard else 'Hide'

            else:
                boardBtn.alpha = 0.5




        elif upFingers[1] and not upFingers[2]:
            if whiteBoard.isOver(x, y) and not hideBoard:
                #print('index finger is up')
                cv2.circle(frame, positions[8], brushSize, color,-1)
                #drawing on the canvas
                if px == 0 and py == 0:
                    px, py = positions[8]
                if color == (0,0,0):
                    cv2.line(canvas, (px,py), positions[8], color, eraserSize)
                else:
                    cv2.line(canvas, (px,py), positions[8], color,brushSize)
                px, py = positions[8]

        else:
            px, py = 0, 0

    # put colors button
    colorsBtn.drawRect(frame)
    cv2.rectangle(frame, (colorsBtn.x, colorsBtn.y), (colorsBtn.x +colorsBtn.w,
colorsBtn.y+colorsBtn.h), (255,255,255), 2)
```

```
    # put white board buttin
    boardBtn.drawRect(frame)
    cv2.rectangle(frame, (boardBtn.x, boardBtn.y), (boardBtn.x +boardBtn.w,
boardBtn.y+boardBtn.h), (255,255,255), 2)

    #put the white board on the frame
    if not hideBoard:
        whiteBoard.drawRect(frame)
        ########### moving the draw to the main image #########
        canvasGray = cv2.cvtColor(canvas, cv2.COLOR_BGR2GRAY)
        _, imgInv = cv2.threshold(canvasGray, 20, 255, cv2.THRESH_BINARY_INV)
        imgInv = cv2.cvtColor(imgInv, cv2.COLOR_GRAY2BGR)
        frame = cv2.bitwise_and(frame, imgInv)
        frame = cv2.bitwise_or(frame, canvas)


    ########## pen colors' boxes #########
    if not hideColors:
        for c in colors:
            c.drawRect(frame)
            cv2.rectangle(frame, (c.x, c.y), (c.x +c.w, c.y+c.h), (255,255,255), 2)

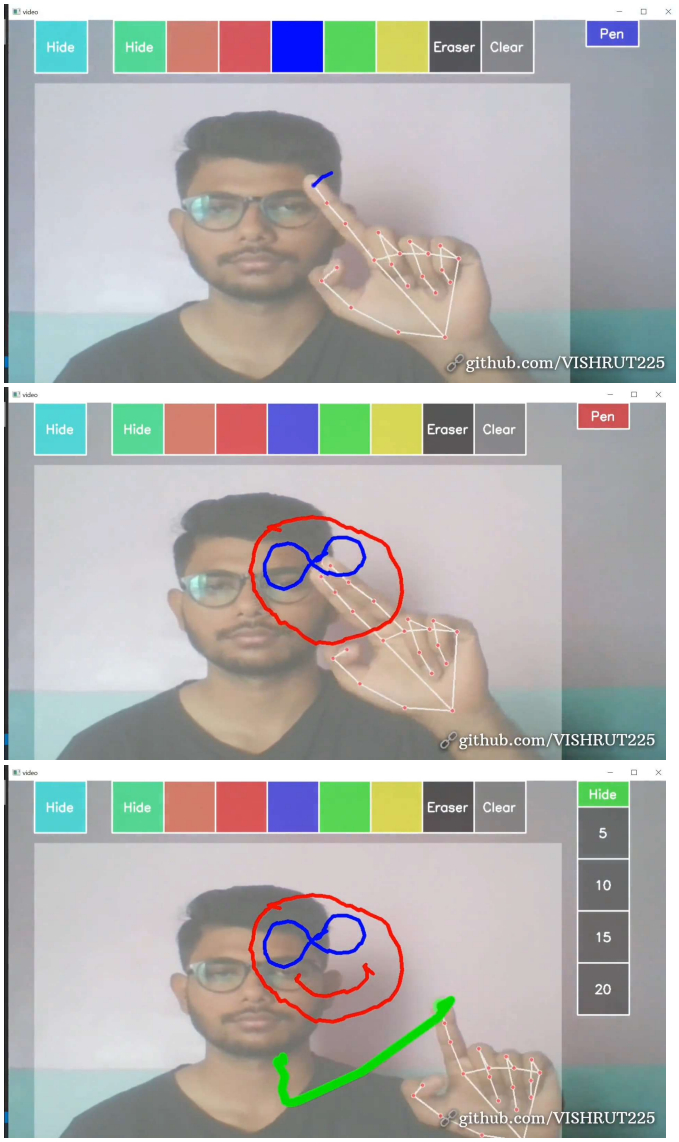
        clear.drawRect(frame)
        cv2.rectangle(frame, (clear.x, clear.y), (clear.x +clear.w, clear.y+clear.h),
(255,255,255), 2)


    ########## brush size boxes ######
    penBtn.color = color
    penBtn.drawRect(frame)
    cv2.rectangle(frame, (penBtn.x, penBtn.y), (penBtn.x +penBtn.w, penBtn.y+penBtn.h),
(255,255,255), 2)
    if not hidePenSizes:
        for pen in pens:
            pen.drawRect(frame)
            cv2.rectangle(frame, (pen.x, pen.y), (pen.x +pen.w, pen.y+pen.h),
(255,255,255), 2)


    cv2.imshow('video', frame)
    #cv2.imshow('canvas', canvas)
    k= cv2.waitKey(1)
    if k == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

4. Demo

A demo showcasing how the application works can be found here.



- **Full Demo Video:** https://drive.google.com/file/d/1OVfLGW_KG4lV4bULOJcJvphYkDXzim4p/view?usp=sharing
- **Github Link**: https://github.com/VISHRUT225/Hand-Sketch

---

5. Conclusion

Hand-Sketch demonstrates the potential of computer vision in creating natural user interfaces. By leveraging real-time hand detection and gesture recognition, it allows users to interact with a virtual whiteboard without needing any additional hardware other than a camera.